

Wifi-seacurity analyzer

PROJECT MEMBERS:

SADAF IQBAL 23-ARID-740

TEHNIAT SAQIB 23-ARID-758

SUBMITTED TO : DR.MEEMONA

Documentation: Wi-Fi Security Analyzer

Table of Contents

1. Introduction
 2. Features
 3. Requirements
 4. Implementation
 5. Function Descriptions
 6. Input/Output Flow
 7. File Structure
 8. Future Enhancements
-

1. Introduction

The **Wi-Fi Security Analyzer** is a C++ application that allows users to manage, monitor, and analyze Wi-Fi networks. It provides functionalities for adding networks, removing them, updating their signal strength, and assessing their security based on encryption types. The program is designed to help users maintain secure and efficient Wi-Fi infrastructure.

2. Features

- **Add Wi-Fi Networks:** Input details such as SSID, MAC Address, Signal Strength, and Security Type.
 - **Remove Wi-Fi Networks:** Remove an existing network by its SSID.
 - **Update Signal Strength:** Simulate real-world changes in signal strength.
 - **Display Networks:** View all the stored Wi-Fi networks with their details.
 - **Analyze Network Security:** Evaluate the security of networks and provide actionable recommendations.
 - **Save Report:** Save network details and analysis to a text file (`WiFiSecurityReport.txt`).
 - **Interactive Menu:** User-friendly console interface for easy navigation.
-

3. Requirements

- **Programming Language:** C++
 - **Compiler:** Any C++ compiler (e.g., GCC, MSVC, Clang)
 - **Operating System:** Windows, macOS, or Linux
-

4. Implementation

Data Structure

The `WiFiNetwork` struct is used to represent a Wi-Fi network, with attributes:

- `ssid`: The network name.
- `macAddress`: Unique MAC address.
- `signalStrength`: Signal strength in dBm (always negative values).
- `securityType`: Encryption type (e.g., Open, WPA2, etc.).

Menu Design

The main program operates through an interactive menu where users can perform various actions. The menu offers the following options:

1. Add Wi-Fi Network
 2. Remove Wi-Fi Network
 3. Update Signal Strength
 4. Display Networks
 5. Analyze Network Security
 6. Save Report to File
 7. Exit
-

5. Function Descriptions

5.1 `void displayMenu()`

Displays the available options to the user.

5.2 `void addNetwork(std::vector<WiFiNetwork>& networks)`

Prompts the user to enter details of a new Wi-Fi network and stores it in the list.

5.3 `void removeNetwork(std::vector<WiFiNetwork>& networks)`

Allows the user to remove a Wi-Fi network by entering its SSID.

5.4 `void updateSignalStrength(std::vector<WiFiNetwork>& networks)`

Simulates real-world signal strength fluctuations by randomly adjusting the signal strength of all stored networks.

5.5 `void displayNetworks(const std::vector<WiFiNetwork>& networks)`

Displays all stored Wi-Fi networks along with their details.

5.6 `void analyzeNetworkSecurity(const std::vector<WiFiNetwork>& networks)`

Analyzes the security of each network based on its encryption type and provides recommendations.

5.7 `void saveReportToFile(const std::vector<WiFiNetwork>& networks)`

Saves the details of all networks and their analysis to a file named `WiFiSecurityReport.txt`.

5.8 `int main()`

The entry point of the program that controls the execution flow based on user input.

6. Input/Output Flow

Input

- Users interact with the program by selecting options from the menu.
- Inputs required for each menu option:
 - **Add Network:** SSID, MAC Address, Signal Strength, Security Type.
 - **Remove Network:** SSID of the network to be removed.
 - **Analyze Security:** No additional input.

Output

- Displays detailed information about networks.
- Security analysis for each network, including risks and recommendations.
- Saves network details and analysis in `WiFiSecurityReport.txt`.

Sample Input/Output

Example 1: Adding a Network

```
mathematica
CopyEdit
Enter SSID: MyWiFi
Enter MAC Address: 00:1A:2B:3C:4D:5E
Enter Signal Strength (in dBm): -65
Enter Security Type (Open, WEP, WPA, WPA2, WPA3): WPA2
Network added successfully!
```

Example 2: Security Analysis Output

```
yaml
CopyEdit
SSID: MyWiFi
- Good Security: WPA2 is secure.
  Recommendation: Ensure strong passwords and regular updates.
```

Example 3: Report Saved

```
css
CopyEdit
Report saved to WiFiSecurityReport.txt
```

7. File Structure

The program saves a detailed report in a file named `WiFiSecurityReport.txt`. The file contains:

1. SSID
 2. MAC Address
 3. Signal Strength
 4. Security Type
 5. Security Recommendations
-

8. Future Enhancements

The program can be extended with the following features:

1. **Real Packet Sniffing:** Integrate libraries like `pcap` for real-time packet analysis.
2. **Heatmap Generation:** Generate visual maps for signal strength.
3. **Network Penetration Testing:** Add tools to mimic real-world attack scenarios.
4. **Password Strength Analysis:** Assess the quality of network passwords.
5. **GUI Integration:** Create a graphical user interface for better usability.

CODE:

```
#include <iostream>
#include <vector>
#include <string>
#include <fstream>
#include <cstdlib>
#include <ctime>

// Structure to represent a Wi-Fi network
struct WiFiNetwork {
    std::string ssid;
    std::string macAddress;
    int signalStrength; // Signal strength in dBm
    std::string securityType; // Encryption type: Open, WEP, WPA, WPA2, WPA3
};

// Function prototypes
void displayMenu();
void addNetwork(std::vector<WiFiNetwork>& networks);
void removeNetwork(std::vector<WiFiNetwork>& networks);
void updateSignalStrength(std::vector<WiFiNetwork>& networks);
void displayNetworks(const std::vector<WiFiNetwork>& networks);
void analyzeNetworkSecurity(const std::vector<WiFiNetwork>& networks);
void saveReportToFile(const std::vector<WiFiNetwork>& networks);

// Function to display the menu
void displayMenu() {
    std::cout << "\nWi-Fi Security Analyzer Menu:\n";
    std::cout << "1. Add Wi-Fi Network\n";
    std::cout << "2. Remove Wi-Fi Network\n";
    std::cout << "3. Update Signal Strength\n";
    std::cout << "4. Display Networks\n";
    std::cout << "5. Analyze Network Security\n";
    std::cout << "6. Save Report to File\n";
    std::cout << "7. Exit\n";
    std::cout << "Enter your choice: ";
}
```

```

- }

// Function to add a Wi-Fi network
] void addNetwork(std::vector<WiFiNetwork>& networks) {
    WiFiNetwork newNetwork;
    std::cout << "Enter SSID: ";
    std::cin >> newNetwork.ssid;
    std::cout << "Enter MAC Address: ";
    std::cin >> newNetwork.macAddress;
    std::cout << "Enter Signal Strength (in dBm): ";
    std::cin >> newNetwork.signalStrength;
    std::cout << "Enter Security Type (Open, WEP, WPA, WPA2, WPA3): ";
    std::cin >> newNetwork.securityType;
    networks.push_back(newNetwork);
    std::cout << "Network added successfully!\n";
- }

// Function to remove a Wi-Fi network
] void removeNetwork(std::vector<WiFiNetwork>& networks) {
    std::string ssid;
    std::cout << "Enter the SSID of the network to remove: ";
    std::cin >> ssid;

    for (auto it = networks.begin(); it != networks.end(); ++it) {
    ]     if (it->ssid == ssid) {
        networks.erase(it);
        std::cout << "Network removed successfully!\n";
        return;
    ]     }
    }
    std::cout << "Network not found!\n";
- }

// Function to update signal strength dynamically
] void updateSignalStrength(std::vector<WiFiNetwork>& networks) {
    std::srand(std::time(0));

```

```

    for (auto it = networks.begin(); it != networks.end(); ++it) {
        if (it->ssid == ssid) {
            networks.erase(it);
            std::cout << "Network removed successfully!\n";
            return;
        }
    }
    std::cout << "Network not found!\n";

// Function to update signal strength dynamically
    void updateSignalStrength(std::vector<WiFiNetwork>& networks) {
        std::srand(std::time(0));
        for (auto& network : networks) {
            network.signalStrength += (std::rand() % 11 - 5); // Random fluctuation (-5 to +5)
            if (network.signalStrength > 0) {
                network.signalStrength = 0; // Ensure signal strength remains negative
            }
        }
        std::cout << "Signal strengths updated successfully!\n";

// Function to display the detected Wi-Fi networks
        void displayNetworks(const std::vector<WiFiNetwork>& networks) {
            if (networks.empty()) {
                std::cout << "No Wi-Fi networks detected!\n";
                return;
            }

            std::cout << "\nDetected Wi-Fi Networks:\n";
            std::cout << "-----\n";
            for (const auto& network : networks) {
                std::cout << "SSID: " << network.ssid << "\n";
                std::cout << "MAC Address: " << network.macAddress << "\n";
                std::cout << "Signal Strength: " << network.signalStrength << " dBm\n";
            }
        }
    }

```

```

        std::cout << "MAC Address: " << network.macAddress << "\n";
        std::cout << "Signal Strength: " << network.signalStrength << " dBm\n";
        std::cout << "Security Type: " << network.securityType << "\n";
        std::cout << "-----\n";
    }
}

// Function to analyze network security
void analyzeNetworkSecurity(const std::vector<WiFiNetwork>& networks) {
    if (networks.empty()) {
        std::cout << "No Wi-Fi networks to analyze!\n";
        return;
    }

    std::cout << "\nWi-Fi Security Analysis:\n";
    std::cout << "-----\n";
    for (const auto& network : networks) {
        std::cout << "SSID: " << network.ssid << "\n";
        if (network.securityType == "Open") {
            std::cout << "- Security Risk: Open network! Highly vulnerable to attacks.\n";
            std::cout << "Recommendation: Configure WPA2 or WPA3 encryption.\n";
        } else if (network.securityType == "WEP") {
            std::cout << "- Security Risk: WEP is outdated and insecure.\n";
            std::cout << "Recommendation: Upgrade to WPA2 or WPA3.\n";
        } else if (network.securityType == "WPA") {
            std::cout << "- Moderate Security: WPA is better than WEP but still vulnerable.\n";
            std::cout << "Recommendation: Upgrade to WPA2 or WPA3 for better protection.\n";
        } else if (network.securityType == "WPA2") {
            std::cout << "- Good Security: WPA2 is secure.\n";
            std::cout << "Recommendation: Ensure strong passwords and regular updates.\n";
        } else if (network.securityType == "WPA3") {
            std::cout << "- Excellent Security: WPA3 provides the best encryption.\n";
            std::cout << "Recommendation: No immediate action required.\n";
        }
        std::cout << "-----\n";
    }
}

```

```

        std::cout << "- Moderate Security: WPA is better than WEP but still vulnerable.\n";
        std::cout << "Recommendation: Upgrade to WPA2 or WPA3 for better protection.\n";
    } else if (network.securityType == "WPA2") {
        std::cout << "- Good Security: WPA2 is secure.\n";
        std::cout << "Recommendation: Ensure strong passwords and regular updates.\n";
    } else if (network.securityType == "WPA3") {
        std::cout << "- Excellent Security: WPA3 provides the best encryption.\n";
        std::cout << "Recommendation: No immediate action required.\n";
    }
    std::cout << "-----\n";
}

// Function to save the analysis report to a file
void saveReportToFile(const std::vector<WiFiNetwork>& networks) {
    std::ofstream file("WiFiSecurityReport.txt");

    if (!file) {
        std::cout << "Error: Unable to create report file!\n";
        return;
    }

    file << "Wi-Fi Security Report:\n";
    file << "-----\n";
    for (const auto& network : networks) {
        file << "SSID: " << network.ssid << "\n";
        file << "MAC Address: " << network.macAddress << "\n";
        file << "Signal Strength: " << network.signalStrength << " dBm\n";
        file << "Security Type: " << network.securityType << "\n";
        file << "-----\n";
    }

    file.close();
    std::cout << "Report saved to WiFiSecurityReport.txt\n";
}

```

```

int main() {
    std::vector<WiFiNetwork> networks;
    int choice;

    do {
        displayMenu();
        std::cin >> choice;

        switch (choice) {
            case 1:
                addNetwork(networks);
                break;
            case 2:
                removeNetwork(networks);
                break;
            case 3:
                updateSignalStrength(networks);
                break;
            case 4:
                displayNetworks(networks);
                break;
            case 5:
                analyzeNetworkSecurity(networks);
                break;
            case 6:
                saveReportToFile(networks);
                break;
            case 7:
                std::cout << "Exiting Wi-Fi Security Analyzer. Goodbye!\n";
                break;
            default:
                std::cout << "Invalid choice! Please try again.\n";
        }
    } while (choice != 7);

    break;
    case 7:
        std::cout << "Exiting Wi-Fi Security Analyzer. Goodbye!\n";
        break;
    default:
        std::cout << "Invalid choice! Please try again.\n";
    }
} while (choice != 7);

return 0;
}

```

OUTPUT:

```
Wi-Fi Security Analyzer Menu:
1. Add Wi-Fi Network
2. Remove Wi-Fi Network
3. Update Signal Strength
4. Display Networks
5. Analyze Network Security
6. Save Report to File
7. Exit
Enter your choice: 1
Enter SSID: mynetwork
Enter MAC Address: 00:1A:2B:3C:4D
Enter Signal Strength (in dBm): -45
Enter Security Type (Open, WEP, WPA, WPA2, WPA3): wpa2
Network added successfully!
```

```
Enter your choice: 3
Signal strengths updated successfully!
```

```
Wi-Fi Security Analyzer Menu:
1. Add Wi-Fi Network
2. Remove Wi-Fi Network
3. Update Signal Strength
4. Display Networks
5. Analyze Network Security
6. Save Report to File
7. Exit
Enter your choice: 4
```

```
Detected Wi-Fi Networks:
```

```
-----
SSID: mynetwork
MAC Address: 00:1A:2B:3C:4D
Signal Strength: -44 dBm
Security Type: wpa2
-----
```

```
Wi-Fi Security Analyzer Menu:
1. Add Wi-Fi Network
2. Remove Wi-Fi Network
3. Update Signal Strength
4. Display Networks
5. Analyze Network Security
6. Save Report to File
7. Exit
```

Enter your choice: 5

Wi-Fi Security Analysis:

SSID: mynetwork

Wi-Fi Security Analyzer Menu:

1. Add Wi-Fi Network
2. Remove Wi-Fi Network
3. Update Signal Strength
4. Display Networks
5. Analyze Network Security
6. Save Report to File
7. Exit

Enter your choice: 6

Report saved to WiFiSecurityReport.txt

Wi-Fi Security Analyzer Menu:

1. Add Wi-Fi Network
2. Remove Wi-Fi Network
3. Update Signal Strength
4. Display Networks
5. Analyze Network Security
6. Save Report to File
7. Exit

Wi-Fi Security Analyzer Menu:

1. Add Wi-Fi Network
2. Remove Wi-Fi Network
3. Update Signal Strength
4. Display Networks
5. Analyze Network Security
6. Save Report to File
7. Exit

Enter your choice: 2

Enter the SSID of the network to remove: mynetwork

Network removed successfully!

Wi-Fi Security Analyzer Menu:

1. Add Wi-Fi Network
2. Remove Wi-Fi Network
3. Update Signal Strength
4. Display Networks
5. Analyze Network Security
6. Save Report to File
7. Exit

Enter your choice: 7

Exiting Wi-Fi Security Analyzer. Goodbye!

Process exited after 238.3 seconds with return value 0

Press any key to continue . . .